CSS Notes Table of Contents 📌

1️⃣ **CSS Selectors**

- Universal Selector
- Element Selector
- ID Selector
- Class Selector
- Group Selector
- Child Selector
- Descendant Selector
- Pseudo Selector
- Pseudo Child Selector
- Pseudo nth-Child Selector
- Additional Selectors

2️⃣ **CSS Specificity**

- Specificity Calculation
- !important Flag

3️⃣ **CSS Box Model**

- Dimension Calculation
- Box Sizing

4️⃣ **CSS Fonts**

- Generic Font Families
- Custom Fonts
- Font Properties

5️⃣ **CSS Shadows and Outline**

- Box Shadows

- Outline Styling

6️⃣ **CSS List Styling**

- List Marker Customization

7️⃣ **CSS Overflow**

- Handling Overflow Content

8️⃣ **CSS Position**

- Static, Relative, Absolute, Fixed, Sticky
- z-index

9️⃣ **CSS Variables**

- Defining and Using Variables

🔟 **CSS Media Queries**

- Responsive Design

1️⃣1️⃣ **CSS Flexbox**

- Flex Direction, Axis, and Alignment
- Gap Property
- Order Property

1️⃣2️⃣ **CSS Grid**

- Grid Rows & Columns
- Grid Template Properties

1️⃣3️⃣ **CSS Transform**

- Rotation, Scaling, Skewing, Translation

1️⃣4️⃣ **CSS Transition**

- Smooth Property Transitions

15 CSS Animation

- Keyframes and Animation Properties

16 CSS Object

- Background and Object Positioning

📌 CSS Selectors 🎯

CSS selectors are used to target HTML elements and apply styles. Here's a breakdown of the different types of selectors:

🔹 Universal Selector (`*`)

```
* {
  property: value;
}
```

👉 **Targets all elements** on the page and applies the given styles.

📝 **Tip:** Be careful when using this selector as it affects every element, which may lead to performance issues in large applications.

🔹 Element Selector

```
p {
  property: value;
}
```

👉 Targets all elements of a specific type (e.g., `<p>` for paragraphs).

📝 **Tip:** Use this for global styling but **avoid overuse** as it applies styles to all elements of that type.

🔹 ID Selector (`#id`)

```
#myID {
```

```
  property: value;
}
```

👉 Targets an element with a **specific ID**.

📝 **Tip: Avoid using IDs for styling**, as they have high specificity and can be difficult to override. Prefer **classes** instead.

🔹 Class Selector (`.class`)

```
.myClass {
  property: value;
}
```

👉 Targets all elements with the given class.

📝 **Tip:** Classes are **preferred over IDs** because they are reusable and allow for more flexible styling.

🔹 Group Selector (`div, p, a`)

```
div, p, a {
  property: value;
}
```

👉 Allows applying styles to multiple elements at once.

📝 **Tip:** Use this to **avoid repeating styles** when multiple elements need the same properties.

🔹 Child Selector (`>`)

```
div > p {
  property: value;
}
```

👉 Targets only **direct children** of an element.

📝 **Tip:** Useful when you want to style only **immediate** children and not deeper nested elements.

🔹 Descendant Selector (Space)

```
div p {
  property: value;
}
```

👉 Targets **all child elements** inside a parent, regardless of depth.

📝 **Tip:** More flexible than child selectors but may lead to **performance issues** if used too broadly.

🔹 Pseudo Selector (`:hover`)

```
a:hover {
  property: value;
}
```

👉 Styles an element when a **specific state** occurs (e.g., when hovered).

📝 **Tip:** Useful for **interactive elements** like buttons, links, and inputs.

🔹 Pseudo Child Selector (`:first-child`)

```
p:first-child {
   property: value;
}
```

👉 Targets the **first child** of a parent element.

📝 **Tip:** Great for making the first item in a list or section stand out.

🔹 Pseudo nth-Child Selector (`:nth-child(n)`)

```
p:nth-child(n) {
  property: value;
}
```

👉 Targets elements based on **position** (e.g., every even or odd child).

📝 **Tip:** Use this for **striped tables** or alternating background colors in lists.

📌 CSS Specificity 🎯

CSS specificity determines which rule is applied when multiple rules target the same element.

📝 Specificity Order

1️⃣ **Inline Styles (`1000`)** – Applied directly in the element (highest priority).
2️⃣ **ID Selector (`100`)** – Unique element styling.

3️⃣ **Class/Attribute/Pseudo-Class (`10`)** – Common styles.
4️⃣ **Element Selector (`1`)** – General styling.
5️⃣ **Universal Selector (`0`)** – Lowest priority.

🔹 Specificity Calculation Example

```
h1 { color: blue; }       /* (1) */
.title { color: red; }    /* (10) */
#heading { color: green; } /* (100) */
```

👉 The `#heading` ID selector **wins** because it has the highest specificity.

📝 **Tip:** Avoid using `!important`, as it **overrides all specificity** and makes debugging difficult.

📌 CSS Box Model 🎯

The CSS box model defines how elements are sized and spaced.

🔹 Box Model Structure 🏗️

```
| Margin (Outside Space) |
| Border (Outline of Element) |
| Padding (Inside Space) |
| Content (Actual Element Content) |
```

👉 **Total Width Calculation:**

Total Width = Width + Left Padding + Right Padding + Left Border + Right Border + Left Margin + Right Margin

👉 **Total Height Calculation:**

Total Height = Height + Top Padding + Bottom Padding + Top Border + Bottom Border + Top Margin + Bottom Margin

---

🔹 Box Sizing (`border-box`)

```
.box {
  box-sizing: border-box;
}
```

👉 Ensures padding & borders are included in the total width/height instead of adding extra space.

📝 **Tip:** Always use `box-sizing: border-box;` for **consistent layouts** and easier spacing control.

📌 CSS Fonts 🎨

Fonts in CSS define the text's appearance, including style, size, and weight.

🔹 Generic Font Families 🖋️

CSS provides five generic font families that act as fallbacks:

1️⃣ **serif** → Fonts with small strokes (e.g., Times New Roman).
2️⃣ **sans-serif** → Clean, modern fonts without strokes (e.g., Arial, Helvetica).
3️⃣ **monospace** → Each character takes equal space (e.g., Courier New).
4️⃣ **cursive** → Handwritten-style fonts (e.g., Pacifico).
5️⃣ **fantasy** → Decorative fonts for special use (e.g., Papyrus).

📝 **Tip:** Always **end font-family with a generic family** to ensure a fallback in case the chosen font isn't available.

🔹 Custom Fonts from Google Fonts 🌐

You can use external fonts like Google Fonts:

✅ **Steps to use Google Fonts:**
1️⃣ Visit Google Fonts.
2️⃣ Select your font and **copy the import link**.
3️⃣ Paste it in your **CSS file**.
4️⃣ Use the font in your styles.

```
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;700&display=
swap');

body {
```

```
  font-family: 'Poppins', sans-serif;
}
```

📝 **Tip:** Google Fonts improve website aesthetics but **too many fonts** can slow down your website. Stick to **2-3 fonts max**.

🔹 Common Font Properties 🔠

```
font-family: 'Arial', sans-serif; /* Sets the font */
font-style: italic; /* Makes text italic */
font-weight: 700; /* Controls boldness */
font-size: 16px; /* Sets text size */
line-height: 1.5; /* Adjusts spacing between lines */
letter-spacing: 1px; /* Adjusts space between characters */
text-transform: uppercase; /* Converts text to uppercase */
text-decoration: underline; /* Adds an underline */
text-align: center; /* Aligns text */
color: #333; /* Sets text color */
```

📝 **Tip:**

- Use `rem` instead of `px` for better **responsiveness**.
- **Avoid excessive letter-spacing** in body text, as it can reduce readability.

📌 CSS Shadows & Outline 🌟

Shadows and outlines add depth and emphasis to elements.

🔹 Box Shadow (`box-shadow`)

box-shadow: h-offset v-offset blur spread color;

👉 Example:

```
.box {
   box-shadow: 5px 5px 10px rgba(0, 0, 0, 0.3);
}
```

✅ **Breakdown:**

- `h-offset` → Horizontal shadow
- `v-offset` → Vertical shadow
- `blur` → Softness of shadow
- `spread` → Size of shadow
- `color` → Shadow color

📝 **Tip:** Use **soft shadows** (`rgba` with low opacity) for a modern look instead of harsh, dark shadows.

🔹 Text Shadow (`text-shadow`)

text-shadow: 2px 2px 5px rgba(0, 0, 0, 0.5);

👉 Creates a glowing or embossed text effect.

📝 **Tip:** Avoid using **too many text shadows** as they can make text **hard to read**.

🔹 Outline (`outline`)

Unlike `border`, **outlines do not take up space** and do not affect layout.

outline: 3px solid red;

📝 **Tip:**

- Outlines **don't impact element dimensions**, making them useful for **focus indicators** (e.g., keyboard navigation).
- **Borders affect layout, but outlines don't** – use outlines for highlighting **without layout shifts**.

📌 CSS List Styling 📝

Lists in CSS can be customized beyond basic bullets.

🔹 List Style Type (`list-style-type`)

```
ul {
  list-style-type: square; /* circle, disc, none */
}
```

👉 Changes the bullet style of lists.

🔹 Custom List Markers 🖼️

✅ Using Images for Bullets

```
ul {
  list-style-image: url('bullet.png');
}
```

✅ Using Emojis as Bullets

```
ul li {
  list-style-type: "✔️";
}
```

📝 **Tip:** Use **list-style-image** for branding purposes or **emoji bullets** to make lists **more visually appealing**.

🔹 List Position (`list-style-position`)

Controls whether the marker is inside or outside the text block.

```
ul {
  list-style-position: inside;
}
```

📝 **Tip:**

- `inside` → Aligns bullet **with text**.
- `outside` (default) → Bullet appears **outside** the text block.
- Use `inside` for **better alignment in narrow layouts**.

📌 CSS Overflow 🚀

Overflow occurs when content **exceeds** the dimensions of its container. CSS provides properties to control how this overflow is handled.

🔹 **Overflow Properties**

```
.container {
  overflow: hidden;  /* Hides overflowing content */
  overflow: scroll;  /* Adds a scrollbar */
  overflow: auto;    /* Adds scrollbar if needed */
  overflow: visible; /* Default: content overflows normally */
}
```

📝 **Tip:** Use `auto` for **responsive designs** to avoid unnecessary scrollbars.

🔹 **Text Overflow**

Handles text that exceeds its container width.

```
.text-box {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;  /* Adds "..." for overflow */
}
```

📝 **Tip:** Use for **headings or UI elements** where truncation improves readability.

📌 CSS Positioning 🎯

CSS `position` controls how elements are placed in the document.

🔹 **Position Values**

✔ `static` (default) – Follows normal document flow
✔ `relative` – Moves relative to its normal position
✔ `absolute` – Moves relative to its nearest positioned ancestor
✔ `fixed` – Stays fixed to the viewport
✔ `sticky` – Acts **relative** until scrolling past a threshold

🔹 **Example Usage**

```
.fixed-header {

  position: fixed;

  top: 0;

  width: 100%;

}
```

📝 **Tip:** Use `fixed` for **navbars** & `sticky` for **headers** that should stay visible while scrolling.

📌 CSS Variables 🎨

Custom properties (**CSS variables**) make styling **more reusable and maintainable**.

🔹 **Declaring Variables**

```
:root {

  --primary-color: #3498db;
```

```
  --font-size: 16px;
}

body {
  color: var(--primary-color);
  font-size: var(--font-size);
}
```

📝 **Tip:** Define variables in `:root` for **global accessibility**.

📌 CSS Media Queries 📱

Media queries make layouts **responsive** based on screen size.

🔹 **Basic Media Query Structure**

```
@media (max-width: 768px) {
  body {
    background-color: lightgray;
  }
}
```

📝 **Tip:** Use `min-width` for **mobile-first** design and `max-width` for **desktop-first** approaches.

1️⃣1️⃣ CSS Flexbox 🚀

Flexbox is a layout model that makes it easy to align items in a **one-dimensional** space—either **row-wise or column-wise**.

📌 **Flex Direction, Axis, and Alignment**

Flexbox aligns elements using the **main axis** (horizontal or vertical) and the **cross axis** (opposite to main).

✅ **flex-direction**: Defines the main axis direction.

✔ `row` (default) – Left to right

✔ `column` – Top to bottom

✔ `row-reverse` – Right to left

✔ `column-reverse` – Bottom to top

📌 **Gap Property**

Defines spacing between flex items:

```
display: flex;

gap: 20px; /* Adds uniform spacing between elements */
```

📝 **Tip:** `gap` works **better than margins** because it prevents collapsing issues.

📌 **Order Property**

Controls the order of flex items without changing HTML.

```
.item-1 {

  order: 2; /* Moves this item to appear after others */

}
```

📝 **Tip:** Use `order` for reordering but keep HTML **logical** for accessibility!

1️⃣2️⃣ CSS Grid 🎯

CSS Grid is a **two-dimensional** layout model, ideal for **complex layouts**.

📌 **Grid Rows & Columns**
Defines structure using rows & columns:

```
.grid-container {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr; /* Three columns */
  grid-template-rows: auto;
}
```

📝 **Tip:** Use `fr` for flexible layouts instead of fixed `px`.

📌 **Grid Template Properties**
Defines reusable grid templates:

```
grid-template-areas:
  "header header"
  "sidebar content"
  "footer footer";
```

📝 **Tip:** Perfect for **semantic, responsive layouts**.

1️⃣3️⃣ CSS Transform 🎭

Allows modifications like **rotation, scaling, skewing, and translation**.

📌 **Rotation, Scaling, Skewing, Translation**

```
.transform-box {
  transform: rotate(45deg) scale(1.5) skew(10deg) translateX(20px);
}
```

📝 **Tip:** Combine multiple transforms for **smooth effects**.

1️⃣4️⃣ CSS Transition 🔄

Enables **smooth animations** between CSS state changes.

📌 **Smooth Property Transitions**

```
.button {
  transition: background-color 0.5s ease-in-out;
}
```

📝 **Tip:** Use `ease-in-out` for **natural effects**.

1️⃣5️⃣ CSS Animation 🎬

Allows **complex, reusable animations**.

📌 **Keyframes and Animation Properties**

```
@keyframes fadeIn {
```

```
  0% { opacity: 0; }
  100% { opacity: 1; }
}

.box {
  animation: fadeIn 2s linear;
}
```

📝 **Tip:** Use animations **sparingly** to avoid performance issues.

1️⃣6️⃣ CSS Object 🖼️

Defines **background positioning & object fit**.

📌 **Background and Object Positioning**

```
.object {
  object-fit: cover;
  background-position: center;
}
```

📝 **Tip:** `object-fit: cover;` ensures images **never get stretched**.